

С++: базовые конструкции, массивы, указатели и файлы

Стартовый конспект по типам данных, операторам, ветвлениям, массивам, указателям, функциям, строкам и файловому вводу-выводу в С++.

Структура материала

1. Типы данных и представление в памяти
2. Операторы, ветвления и управляющие конструкции
3. Массивы, указатели и динамическая память
4. Функции, строки и форматированный вывод
5. Работа с файлами и режимы открытия

Типы данных и представление в памяти

В базовом курсе C++ важно не просто перечислить типы, а понимать, сколько памяти они занимают и как эта память интерпретируется. Тип определяет диапазон значений, возможные операции и ожидания компилятора от переменной.

Для `int` обычно выделяется 4 байта, но рассуждать лучше в терминах битов и диапазонов, а не бытовых аналогий. У знаковых целых один бит используется для кодирования знака, а оставшиеся — для значения по принятой схеме представления.

Тип `char` занимает 1 байт. Для `signed char` типичный диапазон составляет от -128 до 127 , а для `unsigned char` — от 0 до 255. Тип `long long` обычно занимает 8 байт, а тип `long` может отличаться по платформам: на Windows это часто 4 байта, а на Linux и macOS обычно 8 байт.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- `int` в современных компиляторах чаще всего занимает 4 байта.
- `short` обычно занимает 2 байта, `long long` — 8 байт.
- `char` занимает 1 байт; диапазон зависит от знаковости типа.
- Размер `long` зависит от ABI и платформы, поэтому его лучше проверять через `sizeof`.
- `sizeof` помогает сверить предположения о размере типа в конкретной среде.

Проверка размеров типов в конкретной среде

```
#include <iostream>

int main() {
    std::cout << "char: " << sizeof(char) << "\n";
    std::cout << "short: " << sizeof(short) << "\n";
    std::cout << "int: " << sizeof(int) << "\n";
    std::cout << "long: " << sizeof(long) << "\n";
    std::cout << "long long: " << sizeof(long long) << "\n";
}
```

Операторы, ветвления и управляющие конструкции

После типов следующий слой языка — операции над значениями. В учебной практике нужно уверенно различать присваивание, арифметику, инкремент, сравнение и логические связки, потому что на них строятся почти все дальнейшие конструкции.

Условный оператор `if` позволяет выполнить блок кода только при истинности условия, а полная форма `if/else` задаёт два альтернативных сценария. Для коротких выражений иногда используют тернарный оператор, но он должен оставаться читаемым.

Оператор `switch` удобен там, где поведение зависит от фиксированного набора вариантов, например от символа команды или кода режима. При его использовании важно помнить о `break`, иначе управление 'провалится' в следующий `case`.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Логическое НЕ обозначается `!`, конъюнкция — `&&`, дизъюнкция — `||`.
- Тернарный оператор подходит для короткого выбора между двумя значениями.
- `switch` удобен для дискретного выбора по ограниченному набору вариантов.
- Понятность условия важнее краткости записи.

Массивы, указатели и динамическая память

Массив хранит однотипные элементы в непрерывной области памяти и обеспечивает доступ по индексу. В статическом массиве размер известен при объявлении, а в динамическом память выделяется во время выполнения.

Указатель хранит адрес объекта в памяти. Через операцию взятия адреса & можно получить указатель на переменную, а через разыменование * — обратиться к значению по адресу.

Динамический массив создают через new, но в современном C++ чаще предпочитают контейнеры стандартной библиотеки, например std::vector. Тем не менее базовая тема указателей остаётся обязательной, потому что она объясняет адресацию, передачу данных и модель памяти.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Имя массива связано с адресом первого элемента, но массив и указатель не тождественны.
- Нужно различать адрес переменной и значение, которое по этому адресу хранится.
- При динамическом выделении памяти ответственность за освобождение лежит на программе.
- Для учебного кода полезно сначала понять new и delete, а затем перейти к vector и smart pointers.

Простой пример указателя и динамического массива

```
#include <iostream>

int main() {
    int value = 100;
    int* ptr = &value;

    int size = 5;
    int* arr = new int[size]{1, 2, 3, 4, 5};

    std::cout << *ptr << "\n";
    std::cout << arr[2] << "\n";

    delete[] arr;
}
```

Функции, строки и форматированный вывод

Функция помогает вынести повторяемую логику в отдельный блок с именем, параметрами и возвращаемым типом. Это делает программу читаемее и облегчает тестирование небольших фрагментов кода.

В базовом курсе часто показывают строки через массивы символов и функции семейства `printf/scanf`. Это полезно для понимания C-совместимого уровня, но параллельно стоит помнить о `std::string` как более безопасной и удобной абстракции.

Форматированный вывод через `printf` и потоки ввода-вывода через `std::cout` решают похожую задачу разными способами. Важно уметь читать оба варианта, потому что они регулярно встречаются в учебных примерах.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Функция описывается возвращаемым типом, именем, параметрами и телом.
- `std::string` удобнее для современных программ, чем голые символьные массивы.
- Форматные спецификаторы вроде `%d`, `%f`, `%c` и `%s` относятся к `printf`-стилю.
- Для вещественных чисел полезно управлять точностью вывода через `iomanip`.

Работа с файлами и режимы открытия

Файловый ввод-вывод в C++ обычно строится на потоках `ifstream`, `ofstream` и `fstream`. Они позволяют читать, записывать и дописывать данные, а также управлять позицией в файле.

Режимы `ios::in`, `ios::out`, `ios::app`, `ios::trunc`, `ios::binary` и `ios::ate` задают способ открытия файла. Их можно комбинировать, если программе одновременно нужны чтение и запись или запись в конец существующего файла.

При чтении данных важно не полагаться только на `eof()`. Надёжнее строить цикл вокруг самой операции чтения: это уменьшает риск лишней итерации и лучше отражает реальное состояние потока.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- `ios::in` открывает файл для чтения, `ios::out` — для записи.
- `ios::app` дописывает данные в конец файла без перезаписи начала.
- `ios::trunc` очищает существующий файл перед записью.
- `seekg`, `seekp`, `tellg` и `tellp` позволяют работать с позициями в потоке.
- Для чтения по строкам удобнее писать `while (std::getline(file, line))`.

Безопасное чтение файла построчно

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream file("notes.txt");
    std::string line;

    while (std::getline(file, line)) {
        std::cout << line << "\n";
    }
}
```

Материал подготовлен в обезличенном виде на основе локальных учебных заметок и приведён к единому формату для SMHUB.